

Lab 3: Web API Using Custom Model Class, Authorization Filter, and Exception Filter

Hands-on in this document

Objective

To create a Web API using a custom model class, implement CRUD operations, create a custom authorization filter to validate request headers, and create a custom exception filter to log exceptions and return appropriate HTTP status codes.

Part 1: Web API using Custom Model Class

Step 1: Created Model Classes

Created the following model classes inside the Models folder:

- Employee
- Department
- Skill

The Employee model contains the following properties:

- Id
- Name
- Salary
- Permanent
- Department
- Skills
- DateOfBirth

Department contains:

- Id
- Name

Skill contains:

- Id
- Name

Step 2: Created EmployeeController

Created a new API controller named EmployeeController.

Configured the controller using:

- ApiController attribute
- Route attribute

Step 3: Created Constructor

Created a constructor inside EmployeeController.

The constructor initializes the employee list by calling the GetStandardEmployeeList() method.

Step 4: Created GetStandardEmployeeList()

Created a private method named GetStandardEmployeeList().

This method returns a list of Employee objects containing sample employee data.

Each employee record includes:

- Employee Id
- Employee Name
- Salary
- Permanent status
- Department details
- Skill details
- Date of Birth

Step 5: Implemented GET Method

Created the GET action method to return the list of employees.

Added the following attribute:

[ProducesResponseType(StatusCodes.Status200OK)]

The GET method returns ActionResult<List>.

Step 6: Implemented GET by Id

Created a GET action method with an Id parameter.

This method searches for an employee using the given Id.

If the employee exists, employee details are returned.

Otherwise, 404 Not Found is returned.

Step 7: Implemented POST Method

Created the POST action method.

The POST method accepts an Employee object from the request body.

The new employee is added to the employee list.

Step 8: Implemented PUT Method

Created the PUT action method.

The PUT method updates an existing employee using the given Id.

If the employee exists, the details are updated.

Otherwise, 404 Not Found is returned.

Step 9: Implemented DELETE Method

Created the DELETE action method.

The DELETE method removes an employee using the given Id.

If the employee exists, it is removed from the list.

Otherwise, 404 Not Found is returned.

Output

The image displays the Swagger UI for 'FirstWebAPI' (v1.0, OAS 3.0) and the output of an API client request.

Swagger UI:

- API Name: FirstWebAPI 1.0 OAS 3.0
- URL: https://localhost:7202/swagger/v1/swagger.json
- Endpoint: DELETE /api/Employee/{id}
- Schema: Department

API Client Output:

- Request URL: https://localhost:7202/api/Employee
- Server response: 200
- Response body (JSON):

```
[{"id": 1, "name": "Roshan", "salary": 30000, "permanent": true, "department": {"id": 101, "name": "IT"}, "skills": [{"id": 1, "name": "C#"}, {"id": 2, "name": "SQL"}], "dateOfBirth": "1999-05-10T00:00:00"}, {"id": 2, "name": "Ravi", "salary": 40000, "permanent": false, "department": {"id": 102, "name": "HR"}, "skills": [{"id": 3, "name": "Java"}], "dateOfBirth": "2000-03-15T00:00:00"}]
```
- Response headers:

```
content-type: application/json; charset=utf-8
date: Wed, 15 Jul 2025 12:31:53 GMT
server: Kestrel
```

Part 2: Custom Authorization Filter

Step 1: Created Filters Folder

Created a folder named Filters.

Step 2: Created CustomAuthFilter

Created a class named CustomAuthFilter.

Inherited the class from ActionFilterAttribute.

Overrode the OnActionExecuting() method.

The filter checks whether the incoming request contains the Authorization header.

Step 3: Authorization Validation

The filter performs the following validations:

- Checks whether the Authorization header is present.
- Checks whether the Authorization header contains the word Bearer.

If the Authorization header is missing, the request returns:

Invalid request - No Auth token

If the Authorization header does not contain Bearer, the request returns:

Invalid request - Token present but Bearer unavailable

Step 4: Applied Filter

Applied the CustomAuthFilter to EmployeeController using the following attribute:

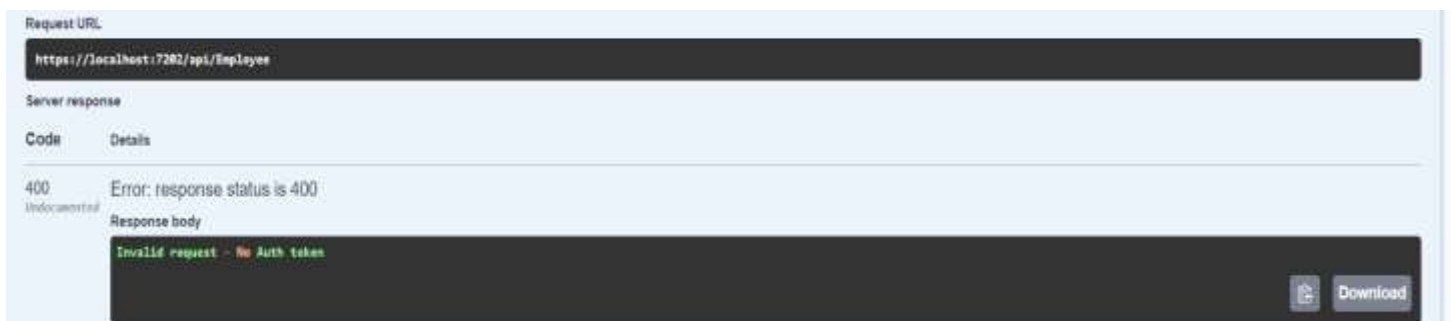
[CustomAuthFilter]

All requests to EmployeeController are validated before executing the action methods.

Output:

Case 1

Authorization header not provided.



Case 2

Authorization header:

Authorization: abc123

Output:

The screenshot shows a REST client interface with the following details:

- Parameters:** No parameters.
- Execute:** A blue button to execute the request.
- Clear:** A button to clear the request.
- Responses:** A section showing the response details.
- Curl:**

```
curl -X 'GET' \
  'https://localhost:7282/api/Employee' \
  -H 'accept: text/plain' \
  -H 'Authorization: abc123'
```
- Request URL:** `https://localhost:7282/api/Employee`
- Server response:** A table with the following entry:

Code	Details
400	Error: response status is 400
- Response body:** `Invalid request - Token present but Bearer unavailable.`
- Download:** A button to download the response body.

Case 3

Authorization header:

Authorization: Bearer abc123

Output:

The screenshot shows a REST client interface with the following details:

- Request URL:** `https://localhost:7282/api/Employee`
- Server response:** A table with the following entry:

Code	Details
200	
- Response body:** A JSON array of employee objects:

```
[
  {
    "id": 1,
    "name": "Rash",
    "salary": 50000,
    "permanent": true,
    "department": {
      "id": 101,
      "name": "IT"
    },
    "skills": [
      {
        "id": 1,
        "name": "C#"
      },
      {
        "id": 2,
        "name": "SQL"
      }
    ],
    "dateOfBIRTH": "1999-05-10T00:00:00"
  },
  {
    "id": 2,
    "name": "Ravi",
    "salary": 40000,
    "permanent": false,
    "department": {
      "id": 102,
      "name": "HR"
    },
    "skills": [
      {
        "id": 3,
        "name": "Java"
      }
    ],
    "dateOfBIRTH": "2000-06-15T00:00:00"
  }
]
```
- Download:** A button to download the response body.

Part 3: Custom Exception Filter

Step 1: Installed NuGet Package

Installed the following NuGet package:

Microsoft.AspNetCore.Mvc.WebApiCompatShim

Step 2: Created CustomExceptionFilter

Created a class named CustomExceptionFilter.

Implemented IExceptionHandler.

Implemented the OnException() method.

Step 3: Exception Logging

Inside the OnException() method:

- Captured exception details.
- Created the folder:

C:\Temp

- Created the file:

ExceptionLog.txt

- Stored the exception details in the log file.

Step 4: Registered Exception Filter

Registered the CustomExceptionFilter globally in Program.cs using:

builder.Services.AddControllers(options =>

```
{  
    options.Filters.Add<CustomExceptionFilter>();  
});
```

Step 5: Modified GET Method

Modified the GET action method to throw an exception for testing.

Added:

[ProducesResponseType(StatusCodes.Status500InternalServerError)]

Step 6: Tested in Swagger

Executed the GET request.

The exception filter handled the exception.

The response returned:

500 Internal Server Error

The exception details were written into:

C:\Temp\ExceptionLog.txt

Output:

Swagger Response:

Example Value | Schema

```
[
  {
    "id": 0,
    "name": "string",
    "salary": 0,
    "permanent": true,
    "department": {
      "id": 0,
      "name": "string"
    },
    "skills": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "dateOfBirth": "2026-07-15T13:10:03.542Z"
  }
]
```

500 Internal Server Error

Response Body:

Request URL

https://localhost:7282/api/Employee

Server response

Code	Details
500	Error: response status is 500

Response body

Internal Server Error

Download

Log File:

C:\Temp\ExceptionLog.txt

```
15-07-2026 18:36:26
System.Exception: Sample Exception
at FirstWebAPI.Controllers.EmployeeController.Get() in C:\Users\AKSHITHA\source\repos\FirstWebAPI\Controllers\EmployeeController.cs:line 87
at lambda_method3(closure, Object, Object[])
at Microsoft.AspNetCore.Mvc.Infrastructure.ActionMethodExecutor.SyncObjectResultExecutor.Execute(ActionContext actionContext, IActionResult
executor, Object controller, Object[] arguments)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeActionMethodAsync()
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Next(State& next, Scope& scope, Object& state, Boolean& isCompleted)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeNextActionFilterAsync()
--- End of stack trace from previous location ---
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Rethrow(ActionExecutedContextSealed context)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Next(State& next, Scope& scope, Object& state, Boolean& isCompleted)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeInnerFilterAsync()
```